

Analyzing the Role of AI-Assisted Programming in Modern Software Development



Name: - K.D.C.S.Nalanda

Index Number: -17359

Registration Number: - 2023s20084

Table Of Contents

- 1.Introduction
2. Task 1 – Conceptual Understanding
 - 2.1 What is AI-Assisted Programming
 - 2.2 Types of AI Tools
 - 2.3 Advantages
 - 2.4 Risks and Limitations
3. Task 2 – Practical Implementation
 - 3.1 Problem Overview (Library Management System)
 - 3.2 Version A – Traditional Programming
 - 3.3 Version B – AI-Assisted Programming
 - 3.4 Comparison of Both Versions
 - 3.5 UML Diagrams
4. Task 3 – Critical Analysis
 - 4.1 Impact of AI on Programming
 - 4.2 Advantages and Disadvantages of AI
 - 4.3 Cases Where AI Should Not Be Used
 - 4.4 Ethical Considerations
5. Task 4 – Conclusion
6. References
7. GitHub Repository Link

1.Introduction

The use of Artificial Intelligence (AI) has become one of the key components in modern software engineering. AI-driven programming solutions are extensively used nowadays to help developers write, debug and optimize their code. The usage of these solutions increases developer productivity due to reduction of efforts and provision of intelligent suggestions during development process.

During the process of coding in a conventional way developers are solely responsible for code design, creation and verification. However, with the appearance of AI-powered solutions like ChatGPT and GitHub Copilot developers have the possibility to produce codes faster and solve complex problems more effectively. This has provided a new way of software development that uses human intelligence combined with artificial intelligence.

At the same time, despite all benefits, AI-powered programming also implies such problems as incorrect code production, lack of understanding, security issues, and ethics concerns. Consequently, it is necessary to consider both pros and cons of the use of AI in programming.

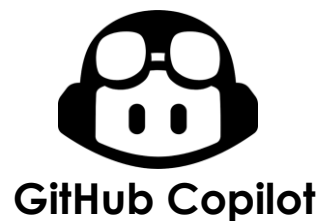
This report is devoted to the development of the Library Management System based on Java. It will be created in two different ways – conventional and AI-powered. The aim is to compare both methods in terms of development process, code quality, and efficiency, and to evaluate the impact of AI on modern programming practices.

2. Task 1 – Conceptual Understanding

2.1 What is AI-Assisted Programming

AI-Assisted Programming is a way of writing software where Artificial Intelligence tools help developers to write, understand, and improve code. These tools make programming easier by reducing manual effort and helping with coding tasks.

It is based on **Generative AI**, which can create new content such as code and text. Tools like ChatGPT and GitHub Copilot use Large Language Models (LLMs) to understand user instructions and generate useful programming code.



In traditional programming, developers must write everything manually and search for solutions on their own. With AI-assisted programming, developers can get code suggestions, fix errors, and solve problems faster.

AI-assisted programming helps save time and improves productivity, but developers still need to check the AI-generated code to make sure it is correct and safe.

2.2 Types of AI Tools

AI coding tools are programs that help developers write and manage code. They use Artificial Intelligence to make things easier for developers. These tools are very useful in software development today. They help developers work faster and do not have to do much work by hand.

1. Code Generation Tools

Code generation tools make code for developers. The developer tells the tool what they want the program to do and the AI coding tool generates the code.

Example: ChatGPT, GitHub Copilot

2. Code Completion Tools

Code completion tools assist developers when they are typing code. These tools suggest what code to use or even fill it in for them. This makes coding and reduces mistakes in the code.

Example: GitHub Copilot, Tabnine

3. Debugging Tools

Debugging tools help find mistakes in the code. They also suggest ways to fix these mistakes. This makes it easier for developers to fix both big mistakes in the code.

Example: assisted IDE debugging features, ChatGPT

4. Code Refactoring Tools

Code refactoring tools make existing code better. They make the code cleaner and more efficient. The code still does the same thing.

Example: AI-based refactoring features in IDEs

5. Documentation Generation Tools

These tools make it easier for developers to understand code. They automatically generate explanations and comments for the code.

Example: ChatGPT, AI documentation tools

6. Test Case Generation Tools

Test case generation tools create tests for software. These tests check if the software works correctly and does what it is supposed to do.

Example: AI testing assistants in software development tools

AI coding tools like these are very important in software development today. They help developers work faster and make sure the code is good. They also reduce the amount of work developers have to do. AI coding tools, like AI coding tools make software development easier and better.

2.3 Advantages

AI-Assisted Programming is a deal in software development these days. It makes it easier for developers to write and manage code. So, what are the advantages of AI-Assisted Programming? Let's break it down:

➤ Reduced Search Time

One of the things about AI-Assisted Programming is that it saves developers a lot of time. Usually, developers spend a lot of time searching for help with coding on websites like Google or Stack Overflow. With AI tools they get instant answers and code suggestions right in their development environment. This means they can keep coding without stopping.

➤ Increased Productivity

AI-Assisted Programming helps developers get things done faster. It generates code suggests solutions and automates tasks. Some studies show that developers who use tools like GitHub Copilot can finish tasks up to 55% faster than usual.

➤ IDE Integration and Smooth Workflow

AI tools work well with development environments like Visual Studio Code and IntelliJ IDEA. This means developers can get suggestions and help with errors without having to switch between platforms. It keeps them focused. Makes their workflow better.

➤ Improved Code Quality and Integrity

AI tools help developers write code. They suggest coding practices catch errors early and help keep code consistent. Some tools even help with documentation and test cases which makes software more reliable.

➤ Learning and Skill Development

AI-Assisted Programming is really helpful for people who are just starting out. It explains coding concepts gives examples and walks users through problem-solving steps. This makes learning to code faster especially for students and new developers.

➤ Faster Software Development and Modernization

AI tools help developers make software faster and also help update systems. They make it easier to modernize code and help developers turn systems into modern programming languages and frameworks. AI-Assisted Programming is really good, at helping with all of this.

2.4 Risks and Limitations

AI-assisted programming has benefits in software development. It also has several risks and limitations. These need to be considered when using it in Java-based systems.

One major limitation is **hallucinations**. AI tools generate code that looks correct but is actually incorrect. It is not suitable for the problem. In Java development this can lead to errors. It can also lead to methods or non-functional code. This increases debugging effort.

Another important issue is **security risks**. Generated Java code may have vulnerabilities. These include input validation, improper error handling or unsafe coding practices. If such code is used without review it may expose the system to attacks. It may lead to data breaches.

Privacy concerns are significant. Many AI tools operate through cloud-based systems. This means that parts of the project code may be sent to servers. This can be risky for confidential applications.

AI tools depend heavily on their **training data**. This data may be incomplete or outdated. As a result, AI may not always provide solutions. This is especially true, for the Java frameworks, libraries or programming practices.

Another limitation is **lack of contextual understanding**. AI may generate code based on patterns. It may not fully understand the complete system design. This leads to code that does not perfectly match project requirements. For example, a Library Management System.

AI tools generate code for a Library Management System. The code may not fully match the systems requirements. This is because AI may not understand the systems design.

Java-based systems use AI-assisted programming. They need to consider these limitations. They need to review AI-generated code. This ensures that the code is correct and secure.

3. Task 2 – Practical Implementation

3.1 Problem Overview (Library Management System)

The Library Management System is a computer program that helps manage what happens in a library. This includes things like adding books to the library searching for books giving books to people to take home and getting books back, from them. The Library Management System makes it easier for people to do their jobs and gets things done faster.

In this project, the system is developed in Java using two approaches: traditional programming (Version A) and AI-assisted programming (Version B), to compare both methods.

3.2 Version A – Traditional Programming

Version A was developed using the traditional programming approach without any AI tools. The system was designed and implemented manually in Java using Object-Oriented Programming concepts such as classes, objects, and methods. The program is console-based and provides basic library management functions including user registration, login, adding books, issuing books, returning books, and displaying books.

LibraryManagement_VersionA

├── User.java

├── Book.java

├── Library.java

└── LibraryManagementSystem.java

User.java

```
1  class User {
2      String username;
3      String password;
4
5      User(String username, String password) {
6          this.username = username;
7          this.password = password;
8      }
9  }
```


Book.java

```
1  class Book {
2      int id;
3      String name;
4      boolean issued;
5
6      Book(int id, String name) {
7          this.id = id;
8          this.name = name;
9          this.issued = false;
10     }
11 }
```

library.java

```
1  import java.util.ArrayList;
2
3  class Library {
4
5      ArrayList<Book> books = new ArrayList<>();
6
7      void addBook(Book b) {
8          books.add(b);
9      }
10
11     void showBooks() {
12         for (Book b : books) {
13             System.out.println(b.id + " " + b.name +
14                 (b.issued ? " (Issued)" : " (Available)"));
15         }
16     }
17
18     void issueBook(int id) {
19         for (Book b : books) {
20             if (b.id == id && !b.issued) {
21                 b.issued = true;
22                 System.out.println("Book Issued");
23                 return;
24             }
25         }
26         System.out.println("Book Not Available");
27     }
28
29     void returnBook(int id) {
30         for (Book b : books) {
31             if (b.id == id && b.issued) {
32                 b.issued = false;
33                 System.out.println("Book Returned");
34                 return;
35             }
36         }
37         System.out.println("Invalid Return");
38     }
39 }
```

LibraryManagementSystem.java

```
1  import java.util.ArrayList;
2  import java.util.Scanner;
3
4  public class LibraryManagementSystem {
5
6      public static void main(String[] args) {
7
8          Scanner sc = new Scanner(System.in);
9
10         ArrayList<User> users = new ArrayList<>();
11         Library library = new Library();
12
13         library.addBook(new Book(1, "The Lord of the Rings"));
14         library.addBook(new Book(2, "The Little Prince "));
15         library.addBook(new Book(3, "And Then There Were None "));
16         library.addBook(new Book(4, "Harry Potter and the Sorcerer's Stone"));
17
18         while (true) {
19             System.out.println("1.Register");
20             System.out.println("2.Login");
21             System.out.println("3.Exit");
22             int choice = sc.nextInt();
23
24             if (choice == 1) {
25                 System.out.print("Username: ");
26                 String u = sc.next();
27                 System.out.print("Password: ");
28                 String p = sc.next();
29
30                 users.add(new User(u, p));
31                 System.out.println("Registered!");
32             }
33
34             else if (choice == 2) {
35                 System.out.print("Username: ");
36                 String u = sc.next();
37                 System.out.print("Password: ");
38                 String p = sc.next();
39
40                 boolean login = false;
41
42                 for (User user : users) {
43                     if (user.username.equals(u) && user.password.equals(p)) {
44                         login = true;
45                         System.out.println("Login Success!");
46
47                         while (true) {
48
49                             System.out.println("\n--- Library Menu ---");
50                             System.out.println("1.Add Book");
51                             System.out.println("2.Show Books");
52                             System.out.println("3.Issue Book");
53                             System.out.println("4.Return Book");
54                             System.out.println("5.Logout");
```

.....(next)

```

55
56         int c = sc.nextInt();
57         if (c == 1) {
58             System.out.print("Book ID: ");
59             int id = sc.nextInt();
60             System.out.print("Book Name: ");
61             String name = sc.next();
62
63             library.addBook(new Book(id, name));
64         }
65         else if (c == 2) {
66             library.showBooks();
67         }
68         else if (c == 3) {
69             System.out.print("Enter Book ID: ");
70             library.issueBook(sc.nextInt());
71         }
72         else if (c == 4) {
73             System.out.print("Enter Book ID: ");
74             library.returnBook(sc.nextInt());
75         }
76         else if (c == 5) {
77             break;
78         }
79     }
80 }
81
82
83     if (!login) {
84         System.out.println("Invalid Login!");
85     }
86 }
87
88     else if (choice == 3) {
89         break;
90     }
91 }
92
93     sc.close();
94 }
95 }

```

3.3 Version B – AI-Assisted Programming

Version B was made using computers to help with the programming to make the old Version A system better. The main things that Version B does are the same, as Version A. Some things were changed to make the code easier to understand and use. Computers helped make the design better by adding checks to make sure the information put in is correct breaking the code into parts and adding comments to explain what the code does. Version B also has a way to save and get back the information from files, which makes Version B more useful and gets things done faster.

LibraryManagement_VersionB/

|

|— Book.java

|— User.java

|— Library.java

|— FileManager.java

|— Main.java

Used prompt: -

 PROMPT:

Create a Java console-based Library Management System (Version B – AI-assisted improved version).

The system should use Object-Oriented Programming and include the following features:

- "Book management (add, search, update, delete, display)"
- "User registration and login system"
- "Issue and return books"
- "ArrayList for storing data"
- "Proper encapsulation (private variables with getters/setters)"
- "Input validation using exception handling"
- "File handling to save and load books from a file (books.txt)"
- "Clean modular structure using multiple classes"

Required classes:

1. "Book (with id, title, author, issued status)"
2. "User (username, password)"
3. "Library (core logic for book operations)"
4. "FileManager (save/load data using file handling)"
5. "Main (menu-driven interface with login system)"

Requirements:

- "Code must be beginner-friendly but well-structured"
- "Add comments for each class and method"
- "Use proper formatting and readable code style"
- "Must be suitable for a university assignment (Version B – improved vs Version A)"

Output full working Java code separated by file names.

1. Book.java

```
</> Java   
  
import java.io.Serializable;  
  
// Book class represents a Library book  
public class Book implements Serializable {  
  
    private String id;  
    private String title;  
    private String author;  
    private boolean issued;  
  
    // Constructor  
    public Book(String id, String title, String author) {  
        this.id = id;  
        this.title = title;  
        this.author = author;  
        this.issued = false;  
    }  
  
    // Getters  
    public String getId() { return id; }  
    public String getTitle() { return title; }  
    public String getAuthor() { return author; }  
    public boolean isIssued() { return issued; }  
  
    // Getters  
    public String getId() { return id; }  
    public String getTitle() { return title; }  
    public String getAuthor() { return author; }  
    public boolean isIssued() { return issued; }  
  
    // Setters  
    public void setTitle(String title) { this.title = title; }  
    public void setAuthor(String author) { this.author = author; }  
    public void setIssued(boolean issued) { this.issued = issued; }  
  
    @Override  
    public String toString() {  
        return id + " | " + title + " | " + author +  
            " | " + (issued ? "Issued" : "Available");  
    }  
}
```

2. User.java

 Java



```
// User class handles login system
public class User {

    private String username;
    private String password;

    public User(String username, String password) {
        this.username = username;
        this.password = password;
    }

    public String getUsername() { return username; }
    public String getPassword() { return password; }
}
```

3. Library.java

 Java



```
import java.util.ArrayList;

// Library class handles all book operations
public class Library {

    private ArrayList<Book> books = new ArrayList<>();

    // Add book
    public void addBook(Book book) {
        books.add(book);
        System.out.println("Book added successfully!");
    }

    // Search book
    public Book searchBook(String id) {
        for (Book b : books) {
            if (b.getId().equalsIgnoreCase(id)) {
                return b;
            }
        }
        return null;
    }

    // Update book
    public void updateBook(String id, String title, String author) {
        Book b = searchBook(id);
        if (b != null) {
            b.setTitle(title);
            b.setAuthor(author);
            System.out.println("Book updated successfully!");
        } else {
            System.out.println("Book not found!");
        }
    }

    // Delete book
    public void deleteBook(String id) {
        Book b = searchBook(id);
        if (b != null) {
            books.remove(b);
            System.out.println("Book deleted successfully!");
        } else {
            System.out.println("Book not found!");
        }
    }
}
```



```

// Issue book
public void issueBook(String id) {
    Book b = searchBook(id);
    if (b != null && !b.isIssued()) {
        b.setIssued(true);
        System.out.println("Book issued successfully!");
    } else {
        System.out.println("Book not available!");
    }
}

// Return book
public void returnBook(String id) {
    Book b = searchBook(id);
    if (b != null && b.isIssued()) {
        b.setIssued(false);
        System.out.println("Book returned successfully!");
    } else {
        System.out.println("Invalid return!");
    }
}

// Display books
public void displayBooks() {
    if (books.isEmpty()) {
        System.out.println("No books available.");
        return;
    }

    for (Book b : books) {
        System.out.println(b);
    }
}

public ArrayList<Book> getBooks() {
    return books;
}

public void setBooks(ArrayList<Book> books) {
    this.books = books;
}
}

```

4. FileManager.java

Java

```
import java.io.*;
import java.util.ArrayList;

// Handles file saving and loading
public class FileManager {

    private static final String FILE_NAME = "books.txt";

    // Save books to file
    public static void saveBooks(ArrayList<Book> books) {
        try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(FILE_NAME))) {
            out.writeObject(books);
            System.out.println("Data saved successfully!");
        } catch (IOException e) {
            System.out.println("Error saving file: " + e.getMessage());
        }
    }

    // Load books from file
    public static ArrayList<Book> loadBooks() {
        File file = new File(FILE_NAME);

        if (!file.exists()) {
            return new ArrayList<>();
        }

        try (ObjectInputStream in = new ObjectInputStream(new FileInputStream(FILE_NAME))) {
            return (ArrayList<Book>) in.readObject();
        } catch (Exception e) {
            System.out.println("Error loading file. Starting fresh.");
            return new ArrayList<>();
        }
    }
}
```

5. Main.java

Java

```
import java.util.ArrayList;
import java.util.Scanner;

// Main class (menu + login system)
public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        Library library = new Library();

        ArrayList<User> users = new ArrayList<>();

        // Load saved books
        library.setBooks(FileManager.loadBooks());

        while (true) {

            System.out.println("\n=== LOGIN SYSTEM ===");
            System.out.println("1. Register");
            System.out.println("2. Login");
            System.out.println("3. Exit");

            int choice;
```



```

try {
    choice = Integer.parseInt(sc.nextLine());
} catch (Exception e) {
    System.out.println("Invalid input!");
    continue;
}

// REGISTER
if (choice == 1) {
    System.out.print("Username: ");
    String u = sc.nextLine();
    System.out.print("Password: ");
    String p = sc.nextLine();

    users.add(new User(u, p));
    System.out.println("Registered successfully!");
}

// LOGIN
else if (choice == 2) {

    System.out.print("Username: ");
    String u = sc.nextLine();
    System.out.print("Password: ");
    String p = sc.nextLine();

    boolean login = false;

    for (User user : users) {
        if (user.getUsername().equals(u) &&
            user.getPassword().equals(p)) {
            login = true;
            System.out.println("Login successful!");
        }
    }

    // LIBRARY MENU
    while (true) {

        System.out.println("\n=== LIBRARY MENU ===");
        System.out.println("1. Add Book");
        System.out.println("2. Search Book");
        System.out.println("3. Update Book");
        System.out.println("4. Delete Book");
        System.out.println("5. Display Books");
        System.out.println("6. Issue Book");
        System.out.println("7. Return Book");
        System.out.println("8. Save & Logout");

        int c;

        try {
            c = Integer.parseInt(sc.nextLine());
        } catch (Exception e) {
            System.out.println("Invalid input!");
            continue;
        }
    }
}

```

```

        if (c == 1) {
            System.out.print("ID: ");
            String id = sc.nextLine();
            System.out.print("Title: ");
            String title = sc.nextLine();
            System.out.print("Author: ");
            String author = sc.nextLine();

            library.addBook(new Book(id, title, author));
        }

        else if (c == 2) {
            System.out.print("Enter ID: ");
            Book b = library.searchBook(sc.nextLine());
            System.out.println(b != null ? b : "Not found");
        }

        else if (c == 3) {
            System.out.print("ID: ");
            String id = sc.nextLine();
            System.out.print("New Title: ");
            String t = sc.nextLine();
            System.out.print("New Author: ");
            String a = sc.nextLine();

            library.updateBook(id, t, a);
        }

        else if (c == 4) {
            System.out.print("ID: ");
            library.deleteBook(sc.nextLine());
        }

        else if (c == 5) {
            library.displayBooks();
        }

        else if (c == 6) {
            System.out.print("ID: ");
            library.issueBook(sc.nextLine());
        }

        else if (c == 7) {
            System.out.print("ID: ");
            library.returnBook(sc.nextLine());
        }

        else if (c == 8) {
            FileManager.saveBooks(library.getBooks());
            System.out.println("Logged out...");
            break;
        }
    }
}

if (!login) {
    System.out.println("Invalid login!");
}

else if (choice == 3) {
    System.out.println("Exiting system...");
    break;
}

sc.close();
}
}

```

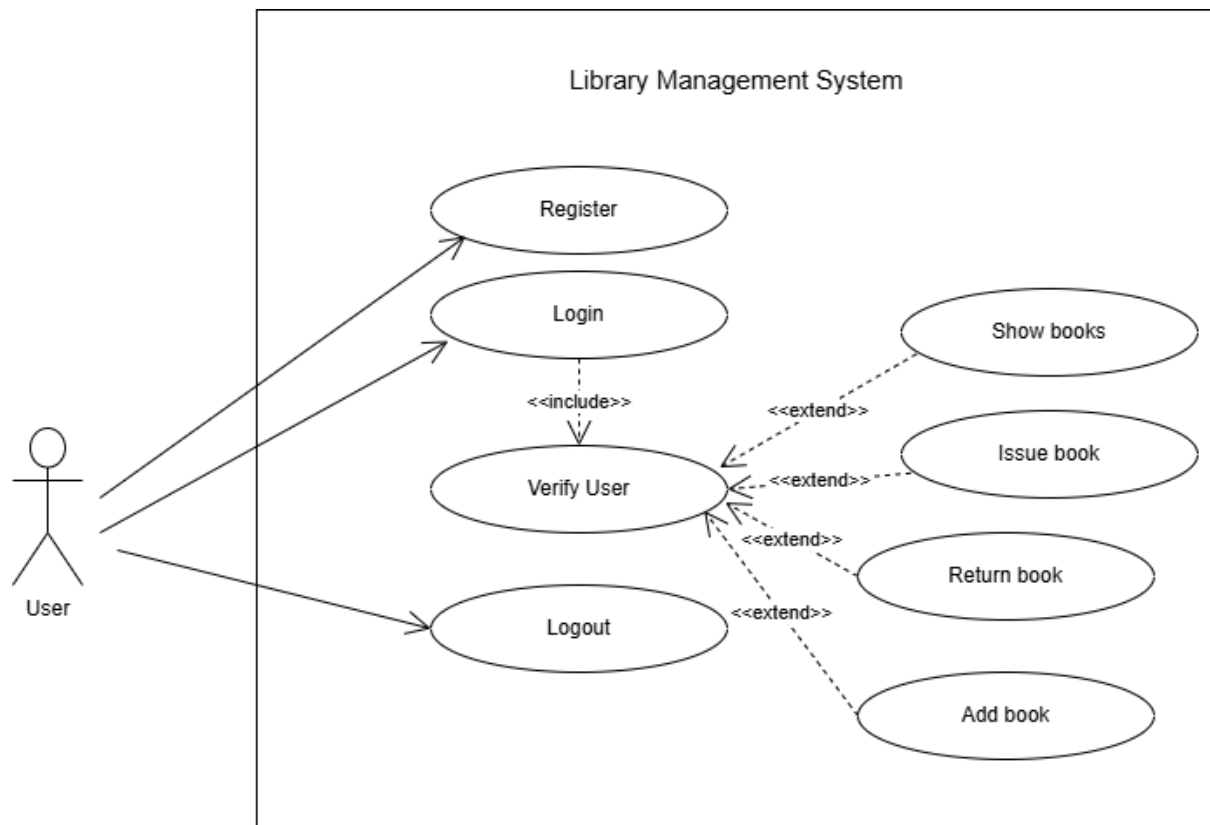
3.4 Comparison of Both Versions

Feature	Version A (Traditional Programming)	Version B (AI-Assisted Programming)
Development	Manually coded	Developed with AI assistance and manual editing (ChatGPT)
Code Structure	Simple	Better organized and modular
Classes	User, Book, Library, LibraryManagementSystem	User, Book, Library, FileManager, Main
Input Validation	Basic	Improved using exception handling
File Handling	Not available	Saves and loads books from books.txt
Data Storage	Temporary	Permanent
Code Readability	Simple	Cleaner with comments and better formatting
User Experience	Basic menu	Improved menu and error messages
Time Taken	About 4 hours	About 1 hour

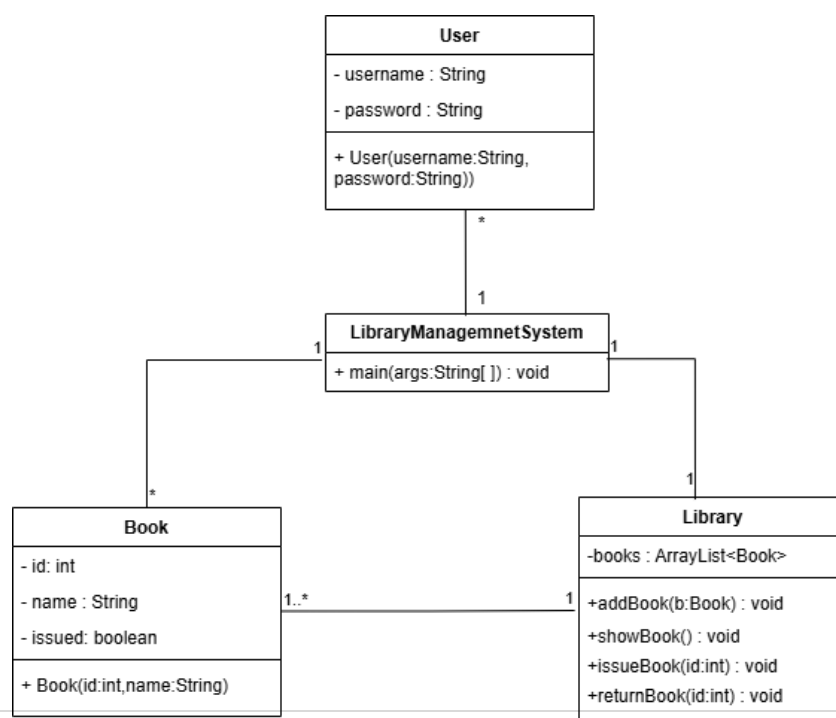
Both Version A and Version B implement the same Java Library Management System using Object-Oriented Programming concepts. Version A was developed manually using traditional programming methods and provides the basic library functions. Version B improves the same system by using AI-assisted suggestions to enhance code structure, input validation, exception handling, and file handling. As a result, Version B is more organized, reliable, and easier to maintain while keeping the same core functionality.

3.5 UML Diagrams

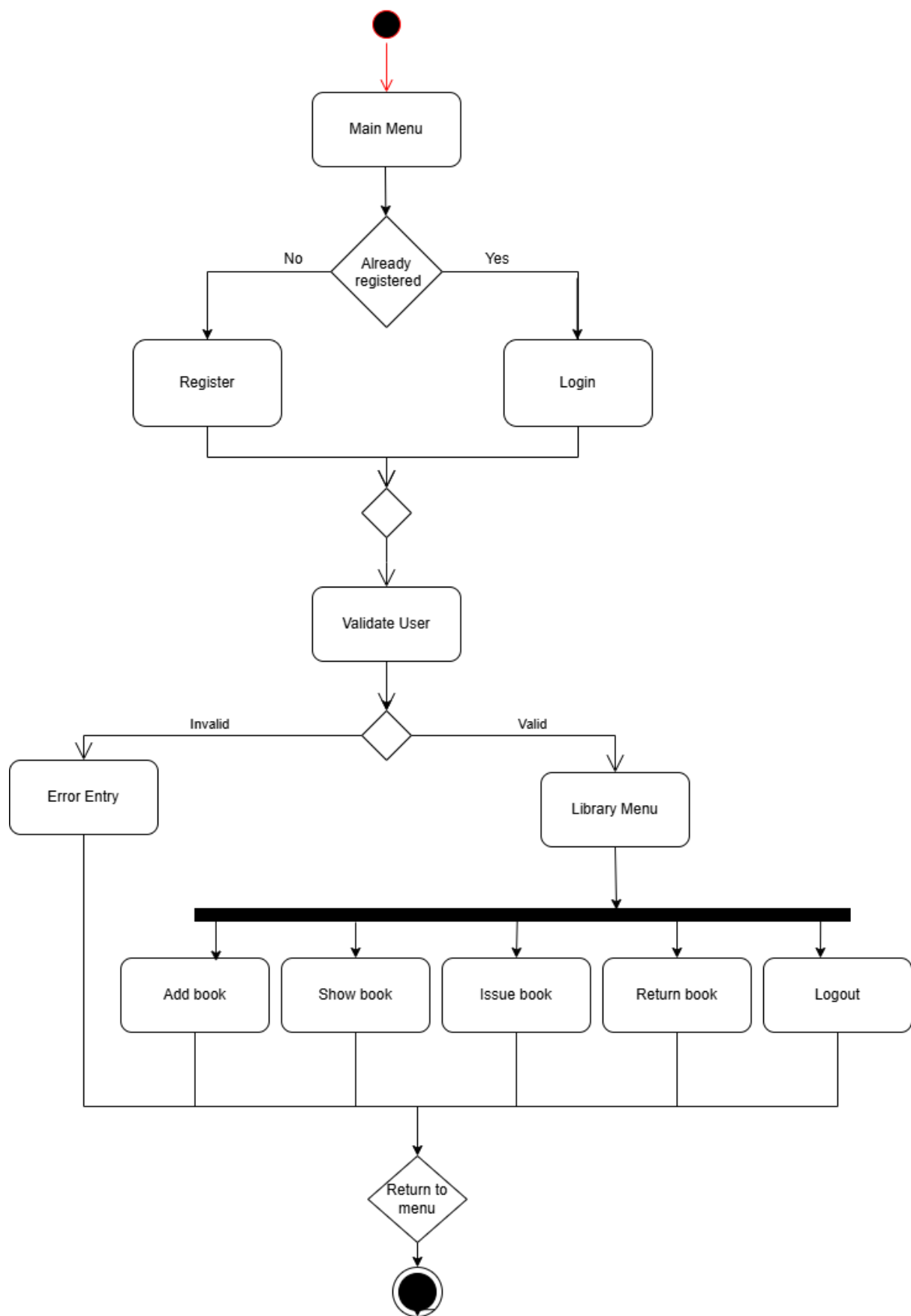
Use case Diagram



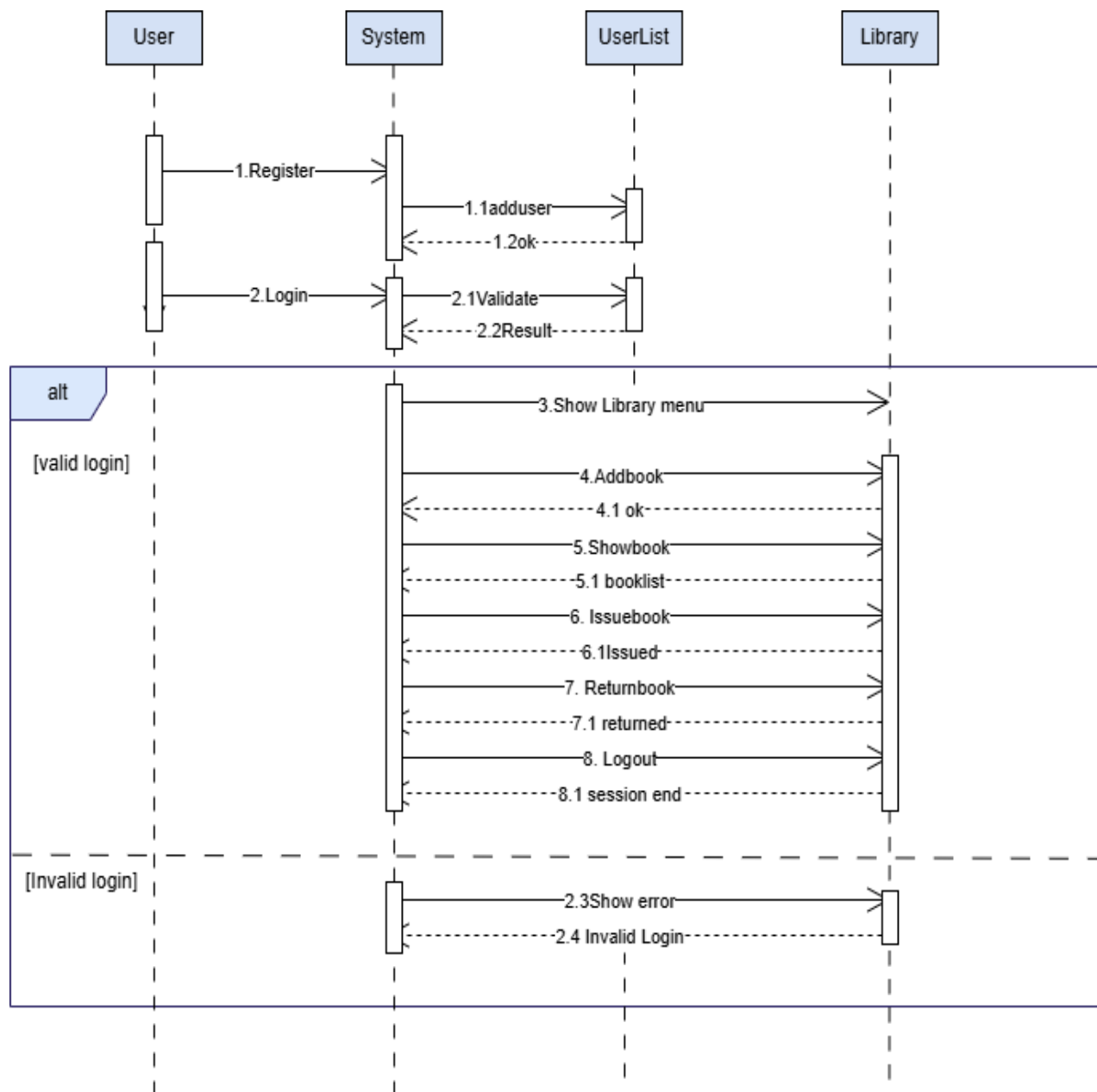
Class Diagram



Activity Diagram



Sequence Diagram



4.Task 3 – Critical Analysis

4.1 Impact of AI on Programming

Artificial Intelligence has significantly changed the way software is developed. It has improved the coding process by assisting programmers in writing code more efficiently. AI tools can generate code, suggest improvements, detect errors, and explain programming concepts. As a result, software development has become faster and more accessible, especially for beginners who are learning programming.

In this project, AI changed my coding approach by helping me design and improve the Library Management System in a more structured and efficient way. It supported me in identifying bugs, improving code readability, and suggesting better design solutions. However, I still had to apply my own programming knowledge to review, test, debug, and modify the code to ensure it met the required functionality. The final AI-assisted version of the system was more organized, easier to understand, and more reliable while maintaining the same functionality as the manually developed version.

4.2 Advantages and Disadvantages of AI

Advantages

- AI reduces the time required for software development.
- It can quickly generate code and provide useful suggestions.
- It helps identify and fix programming errors efficiently.
- AI improves code readability and structure.
- It supports beginners in understanding programming concepts.
- It increases overall developer productivity.

Disadvantages

- AI-generated code may contain mistakes or inefficiencies.
- Over-reliance on AI can reduce problem-solving and coding skills.
- Developers must always review and test AI-generated code.
- AI may not fully understand project requirements or context.
- Some AI tools require internet access or paid subscriptions.

4.3 Cases Where AI Should Not Be Used

Artificial Intelligence should not be used in situations where independent thinking, full understanding, or strict control is required. For example, AI should not be used during exams or assessments where students are expected to demonstrate their own knowledge and problem-solving skills. It should also be avoided when learning basic programming concepts, as relying on AI at an early stage can prevent the development of strong foundational skills.

In addition, AI should not be fully relied upon in critical decision-making systems or sensitive software development, where errors can lead to serious consequences. In such cases, human expertise, careful design, and thorough testing are essential. AI can support the process, but final decisions must always be made by experienced developers.

4.4 Ethical Considerations

I used AI as a supporting tool, not as a replacement for my programming skills and knowledge. In this project, AI helped make the development process easier by identifying bugs, suggesting improvements, and enhancing the overall quality of the code. However, I made the final decisions and used my own programming knowledge to test, debug, modify, and verify the code.

AI does not automatically understand the project requirements or remember previous work. It generates responses based only on the information and prompts provided by the user. Therefore, I carefully reviewed, tested, and verified all AI-generated code before incorporating it into the project.

Using AI responsibly means avoiding plagiarism, being transparent about its use, and ensuring that the final work reflects my own understanding and effort. I treated AI as a learning tool and a productivity aid rather than a substitute for my own thinking and programming skills. By combining my programming knowledge with AI assistance, I was able to develop a more reliable, well-structured, and thoroughly tested Library Management System while maintaining academic integrity.

5.Task 4 – Conclusion

This project successfully developed a Library Management System using Java through both traditional programming and an AI-assisted approach. By comparing these two methods, it was observed that the AI-assisted version was more structured, easier to understand, and more maintainable, while the traditional approach strengthened core programming skills such as object-oriented design, problem-solving, and manual coding ability. Overall, the project demonstrated that combining strong programming knowledge with AI support results in more efficient development and higher-quality software.

From this experience, it is clear that Artificial Intelligence plays a valuable role in both programming education and the software industry. In education, AI helps students learn concepts faster, identify errors, and improve their coding practices, but it should not replace independent thinking, problem-solving skills, or foundational programming knowledge. In the industry, AI improves productivity by assisting in coding, debugging, and optimization, while still requiring human developers to make final decisions and ensure correctness, security, and good design.

In conclusion, AI should be seen as a supportive tool rather than a replacement for a programmer's knowledge and skills. It enhances learning and development, but it cannot replace human understanding, experience, and decision-making. The most effective approach in both education and industry is to combine strong programming fundamentals with the responsible and professional use of Artificial Intelligence.

6. References

Website.<https://www.hkdca.com/wp-content/uploads/2025/06/ai-assisted-programming-oreilly.pdf>

W3Schools. (2025). *Java Tutorial*. Available at: <https://www.w3schools.com/java/>

OpenAI. (2025). *ChatGPT (AI assistance used for code improvement, explanation, and report guidance)*. Available at: <https://chat.openai.com/>

7. GitHub Repository Link:<https://github.com/chanukisewmini1010-sketch/LibraryManagementSystem>